

Informe LaSDPC: Control de temperatura

Calle Huamantínco Luis Eduardo

1. Introducción

El presente informe documenta los cambios realizados en el proyecto LaSDPC para el control de temperatura. Las modificaciones abarcan dos áreas importantes, el simulador del laboratorio y los agentes que razonan sobre los datos del entorno virtual. El objetivo principal fue reemplazar los valores aleatorios de temperatura por valores calculados físicamente a partir del estados de los assets, diferenciando dos ambientes independientes (Datacenter y Laboratorio) y dotar a los agentes de reglas que controlen automáticamente los equipos de enfriamiento en cuanto a la temperatura promedio reportada por los sensores.

2. Contexto

Antes de las modificaciones, el sistema presentaba las siguientes limitaciones:

- Los sensores de temperatura publicaban valores aleatorios
- El simulador calculaba una sola temperatura global sin distinción de zonas
- Los assets de aire acondicionado no tenían efecto sobre la temperatura
- No existían reglas de razonamiento para controlar los ACs

3. Cambios en el simulador de temperatura (Python)

3.1. [simulatorSensor.py](#)

Se modificó la clase TemperatureSensor para que pueda recibir un valor externo de temperatura en lugar de generarlo aleatoriamente.

Se agregó el atributo `_external_value` y el método `set_external_value(value)`.

Cuando `_external_value` tiene un valor asignado, `simulate_reading()` lo retorna en lugar de generar un número aleatorio. Si aún no fue inyectado ningún valor, se mantiene el comportamiento original como fallback.

```
def set_external_value(self, value):
    self._external_value = value

def simulate_reading(self):
    if self._external_value is not None:
        return round(self._external_value, 2)
    return round(random.uniform(20.0,30.0), 2) # fallback
```

3.2. [temperatureSim.py](#)

Este archivo fue refactorizado para soportar múltiples zonas e integrarse en un layout con scroll.

Constructor multi-zona

Se eliminaron los parámetros de centralwidget, gridlayout y grid_row_offset. El constructor ahora recibe parent_widget, parent_layout y zone_name, permitiendo instanciar múltiples simuladores independientes.

Soporte para air conditioning

Se agregaron qAC (3500 W por defecto), numACs y su conteo. En la formula de balance térmico, el AC resta calor al sistema:

```
qNet    = (self.qLamp * self.numLamps) + (self.qHeater *
self.numHeaters) - (self.qAC * self.numACs) + (self.qRack *
self.racks_per_datacenter * self.numDatacenters)

deltaT = (qNet - qLoss) / (self.density * self.volume *
self.cPair)
```

Registro de sensores

Se agrego register_temperature_sensor(sensor_simulator). Cada tick del timer llama a set_external_value(tRoom) en todos los sensores registrados, conectando el simulador con la capa MQTT.

UI con Qframe y QScrollArea

Cada zona construye su propio QFrame con QVBoxLayout interno, colocando dentro de un QScrollArea horizontal debajo del diagrama, maximizando el espacio visual del plano del laboratorio.

Parametros fisicos configurables

qLamp, qHeater, qAC, uValue, wallArea y volume son ahora parametros del constructor, permitiendo valores distintos por zona.

3.3. main8.py

Constantes de zona

Se definieron DATACENTER_ASSET_IDS y DATACENTER_SENSOR_IDS como conjuntos de IDS para determinar la zona de cada elemento sin modificar TestData ni los modelos.

Dos instancias de TemperatureSimulator

Se reemplazó la instanciación de un único simulador por tempSimDataCenter y tempSimLab, ambos dentro de un QScrollArea.

create_sensor_gi y create_asset_gi con asignacion por zona

Ambos métodos fueron modificados para registrar cada elemento en el simulador de zona correspondiente según los conjuntos de IDs definidos.

Simplificacion de addLamp

Se eliminó la actualización manual del contador de lámparas porque updateNumLampsHeatersTurnOn recalcula automáticamente los conteos en cada tick.

4. Cambios en los agentes (Java)

4.1. CollectorAgent.java

El CollectorAgent es el responsable de recolectar los mensajes MQTT de los sensores y assets, y de preparar los datos para enviarlos al ReasonerAgent. Se le agregaron dos cambios clave relacionados con el control térmico por ambiente.

Separación de sensores por ambiente

Se definió el conjunto serverSensors con los ids de los tres sensores del Datacenter (sensor_temperature1, sensor_temperature2, sensor_temperature3). Al iterar sobre los sensores de tipo SensorTemperature, se clasifica cada lectura en una de dos acumulaciones:

- sumServers / countServers: acumula las lecturas de los sensores del Datacenter.
- sumPeople / countPeople: acumula las lecturas del resto de sensores, correspondientes al Laboratorio.

Cálculo de promedios independientes

Al final de cada tick del TicketBehaviour (cada 20 segundos), se calculan dos promedios independientes y se insertan como sensores virtuales en el mapa de sensores antes de enviarlo al ReasonerAgent:

- sensor_temperature_avg_servers
- sensor_temperature_avg_people

Estos sensores virtuales no existen físicamente en el simulador, son contruidos internamente por el CollectorAgent para que el ReasonerAgent pueda razonar sobre ellos con reglas específicas por zona.

4.2. ReasonerAgent.java

El ReasonerAgent recibe los datos del CollectorAgent, los vuelca en la ontología OWL, aplica reglas de razonamiento con Apache Jena y envía los modelos resultantes al ControlAgent. Los cambios se concentraron en las reglas de razonamiento.

Limpieza de instancias AssetAirConditioning y AssetHeat

Se agregadosn dos bloques de limpieza en readOntologyFromFile para eliminar el estado previo (has_state) de los individuos de tipo AssetAirConditioning y AssetHeat antes de repoblar la ontología con los nuevos valores. Esto evita que estados anteriores interfieran con el razonamiento actual.

Nuevas reglas para control térmico por zona

Se agregaron cuatro reglas nuevas al motor de razonamiento para controlar los ACs de cada zona de forma independiente:

Regla	Descripción
r6 (servidores)	Si sensor_temperature_avg_servers > 17.0, encender asset_aicond1, asset_aicond2 y asset_aicond3
r7 (servidores)	Si sensor_temperature_avg_servers <= 17.0, apagar asset_aicond1, asset_aicond2 y asset_aicond3
r8 (laboratorio)	Si sensor_temperature_avg_people > 22.0, encender asset_aicond4 y asset_aicond5
r9 (laboratorio)	Si sensor_temperature_avg_people <= 22.0, apagar asset_aicond4 y asset_aicond5

Estas reglas usan los sensores virtuales generados por el CollectorAgent, por lo que cada zona reacciona unicamente a la temperatura de sus propios sensores. Los umbrales difieren entre zonas: 17.0 para el Datacenter (servidores requieren mayor refrigeracion) y 22.0 para el Laboratorio (temperatura de confort para personas).

Código de las nuevas reglas

```
// Server room temperature control 17°C
"[r6: (eg:sensor_temperature_avg_servers eg:has_value ?v),
greaterThan(?v, 17.0) ->
changeStatePropertyInteger(eg:asset_aircond1, eg:has_state,
1),
changeStatePropertyInteger(eg:asset_aircond2, eg:has_state,
1),
changeStatePropertyInteger(eg:asset_aircond3, eg:has_state,
1)]" +

"[r7: (eg:sensor_temperature_avg_servers eg:has_value ?v),
lessThanOrEqual(?v, 17.0) ->
changeStatePropertyInteger(eg:asset_aircond1, eg:has_state,
0),
```

```

changeStatePropertyInteger(eg:asset_aircond2, eg:has_state,
0),
changeStatePropertyInteger(eg:asset_aircond3, eg:has_state,
0)]" +

// People room temperature control 22°C
"[r8: (eg:sensor_temperature_avg_people eg:has_value ?v),
greaterThan(?v, 22.0) ->
changeStatePropertyInteger(eg:asset_aircond4, eg:has_state,
1),
changeStatePropertyInteger(eg:asset_aircond5, eg:has_state,
1)]" +

"[r9: (eg:sensor_temperature_avg_people eg:has_value ?v),
lessThanOrEqual(?v, 22.0) ->
changeStatePropertyInteger(eg:asset_aircond4, eg:has_state,
0),
changeStatePropertyInteger(eg:asset_aircond5, eg:has_state,
0)]";

```

5. Definición de ambientes

Ambiente	Assets y Sensores asignados
Datacenter	ACs: asset_aircond1, asset_aircond2, asset_aircond3 Lamparas: asset_lamp1 a asset_lamp5 Sensores: sensor_temperature1, sensor_temperature2, sensor_temperature3 Umbral AC: 17°C
Laboratorio	ACs: asset_aircond4, asset_aircond5 Lamparas: asset_lamp6 a asset_lamp15 Sensores: sensor_temperature4 a sensor_temperature7 y sensor_temperature_outside Umbral AC: 22°C

6. Flujo de datos

Con todos los cambios realizados el flujo del sistema es el siguiente:

- Cada segundo, el QTimer de cada TemperatureSimulator calcula la temperatura de su zona en función de los assets activos, considerando que los ACs restan calor.
- La temperatura calculada se inyecta en los TemperatureSensor registrados en esa zona mediante set_external_value(), que los publica vía MQTT cada 5 segundos.

- El CollectorAgent recibe las lecturas MQTT, las clasifica por zona, calcula dos promedios independientes y los adjunta como sensores virtuales al mensaje que envía al ReasonerAgent cada 20 segundos.
- El ReasonerAgent vuelve los datos en la ontología OWL, limpia los estados previos y aplica las reglas de razonamiento.
- El ControlAgent recibe los dos modelos, detecta los cambios producidos por el razonador y publica el nuevo estado de cada AC.
- El simulador recibe esos mensajes MQTT, actualiza el estado visual del asset y lo refleja en la assetList de su zona.

7. Conclusión

Los cambios realizados permitieron transformar el sistema de un simulador con valores aleatorios a un modelo físicamente coherente con dos zonas térmicas independientes. Ahora cada zona calcula su temperatura según sus assets activos, los sensores publican esos valores reales y los agentes razonan sobre promedios por zona para controlar los ACs de forma diferenciada.